Apartment Rentals Bitemporal Database

1. Domain and Why Time Matters

The Apartment Rentals database manages apartments, owners, tenants, property managers, rental agreements, and monthly rents. These facts change over time: a tenant may move into or out of an apartment, a manager may be replaced, and the listed rent may be updated. A conventional database that stores only the latest state cannot reliably reconstruct what was true in the past or what the organization believed at a particular moment.

A question that requires both clocks is: *"What monthly rent did the database believe applied to apartment 1 on 1 August 2026, according to the information available on 1 March 2026?"* The valid-time date identifies the real-world period, while the transaction-time date limits the answer to knowledge available at the specified historical moment.

The rent example demonstrates why this distinction matters. The timeline includes a rent of EUR 975 recorded proactively in advance and a subsequent correction to EUR 950. A query for August 2026 using an earlier transaction-time date may return EUR 975, while a query using a later transaction-time date returns EUR 950. The earlier belief is preserved rather than overwritten. The same principle applies to lease agreements and apartment-manager assignments.

The database is implemented in PostgreSQL 18.6. Its main temporal tables are lease_version, apartment_rent_version, and apartment_manager_version. The tx_registry table records logical changes, while views Q0–Q8 support integrity checking and historical analysis.

2. Modeling Decisions

- *Anchor/version split.* Stable entities are stored in owner, apartment, tenant, and property_manager. Changing facts are kept separately in lease_version, apartment_rent_version, and apartment_manager_version, avoiding duplication of stable data.
- *Sentinel values and intervals.* Open-ended valid-time periods use DATE '9999-12-31', while transaction-time periods use TIMESTAMPTZ '9999-12-31 23:59:59.999999+00'. All intervals follow the half-open convention ([from, to)), allowing adjacent periods without overlap.
- *Time representation.* Valid time uses DATE for day-level precision; transaction time uses TIMESTAMPTZ(6) with UTC. The sim_now() function supports deterministic historical event replay through app.sim_now.
- *Transaction registry.* Each logical change is recorded in tx_registry, linking version rows to a transaction ID, timestamp, author, and reason. A composite foreign key ensures that tx_id and tx_from match the registered transaction.
- *Integrity rules.* Check constraints validate intervals and values, while GiST exclusion constraints prevent conflicting current lease, rent, and manager periods. The v_integrity_violations view provides an additional consistency check; Q0 should return zero rows on the final dataset.

- *Indexes.* As-of and partial indexes support temporal queries. Their actual performance impact must be verified through the Part A EXPLAIN ANALYZE comparison.
- *Advanced tracks.* A1 introduces a write API and triggers to protect temporal history. A2 uses current-state snapshots to demonstrate PostgreSQL 18 valid-time constraints, while retaining the bitemporal history tables.

3. Write Protocol: E3, E4 and E8

The A1 write protocol centralizes modifications to the bitemporal history through five functions: sp_assert_fact, sp_retract_fact, sp_end_fact, sp_correct_fact, and the helper a1_register_transaction. Each operation registers its author, reason, and transaction timestamp in tx_registry.

New facts receive transaction-time metadata automatically. Retractions and corrections close the current version by setting tx_to, preserving the previous record. Corrections insert a replacement version, while sp_end_fact shortens the valid-time interval and creates a replacement with the updated end date.

A shared trigger, a1_guard_history, protects all three temporal tables. It rejects direct modifications, physical deletions, changes to closed versions, and updates that alter fields other than tx_to when closing an open version. A transaction-local flag allows writes performed through the API.

Negative tests attempted direct DELETE and UPDATE operations and produced notices indicating that both were rejected. However, the displayed messages contain a spelling mismatch (forbidden in the trigger versus forbidden in the tests), so the tests did not recognize the exceptions as passes. The trigger blocked the operations, but the test assertions should be corrected before claiming that the negative tests passed.

4. Challenges and Failed Attempts

A central implementation challenge was maintaining two independent time dimensions without overwriting historical information. Updating a fact in place would lose the previous belief, so the protocol closes an old transaction-time period and inserts a replacement. The sentinel convention and half-open intervals also require consistent predicates across the views, constraints, indexes, and test scripts.

The integrity tests deliberately drop exclusion constraints inside a transaction, insert invalid rows, inspect v_integrity_violations, and then roll back. This is a useful demonstration that the integrity view detects modeled violations without leaving test corruption in the final data. The final integrity count should return to zero after rollback.

The A1 advanced script attempts to centralize writes in stored procedures and reject direct history changes through triggers. Its negative tests check that direct DELETE and UPDATE attempts are rejected. However, the API implementation needs a consistency review before it can be considered complete: the table whitelist in a1_insert_fact and related routines must refer to apartment_rent_version, not the mismatched name manager_rent_version. Otherwise, rent operations may fail as unsupported temporal-table operations. In addition, transaction-time

stamping and the closing of historical versions should be tested for every supported table and event type.

A2 addresses a different challenge: applying native valid-time constraints without imposing them incorrectly on the entire bitemporal history. PostgreSQL 18's WITHOUT OVERLAPS is applied to current-state snapshots, because historical transaction-time versions may legitimately overlap in valid time. The snapshots must be rebuilt to reflect later changes; they are not a replacement for the append-only history.

5. Insights and Engine Comparison

Feature	Oracle	SQL Server	PostgreSQL 18
Valid-time	Temporal validity	Requires separate modeling	Native temporal constraints
Transaction-time	Flashback features	Automatic with system-versioned tables	Managed manually
Historical queries	AS OF PERIOD, Flashback queries	FOR SYSTEM_TIME AS OF	Explicit TT predicates
Main Advantage	Mature temporal and historical query features	Automatic system-time versioning	Flexible, explicit modeling with native VT constraints
Main Limitation	Features do not automatically eforce all business rules	VT still needs separate modeling	Availability and exact behavior must be verified against the target release

Results

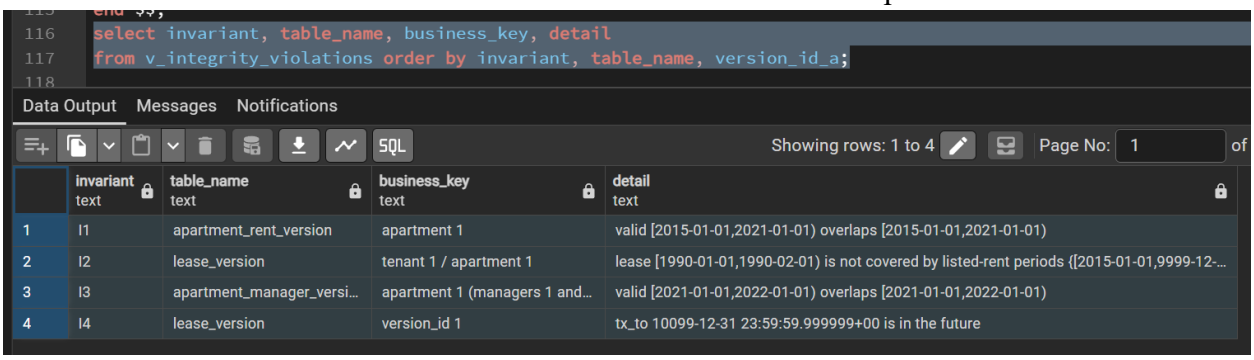
Image 1: Bitemporal Point Query (Q6)

Shows the state of lease, manager and rent facts for a specific valid date and transaction time perspective. It addresses the problem of reconstructing what was valid in reality and what the database knew at a given time.



Image 2: Integrity Violations (Q0)

Display four detected integrity violations (I1-I4), including overlapping rent periods, Alease not covered by rent periods, overlapping manager assignments and an invalid transaction-time end date. It demonstrates how the database detects inconsistencies in temporal data.



```
116  select invariant, table_name, business_key, detail
117  from v_integrity_violations order by invariant, table_name, version_id_a;
```

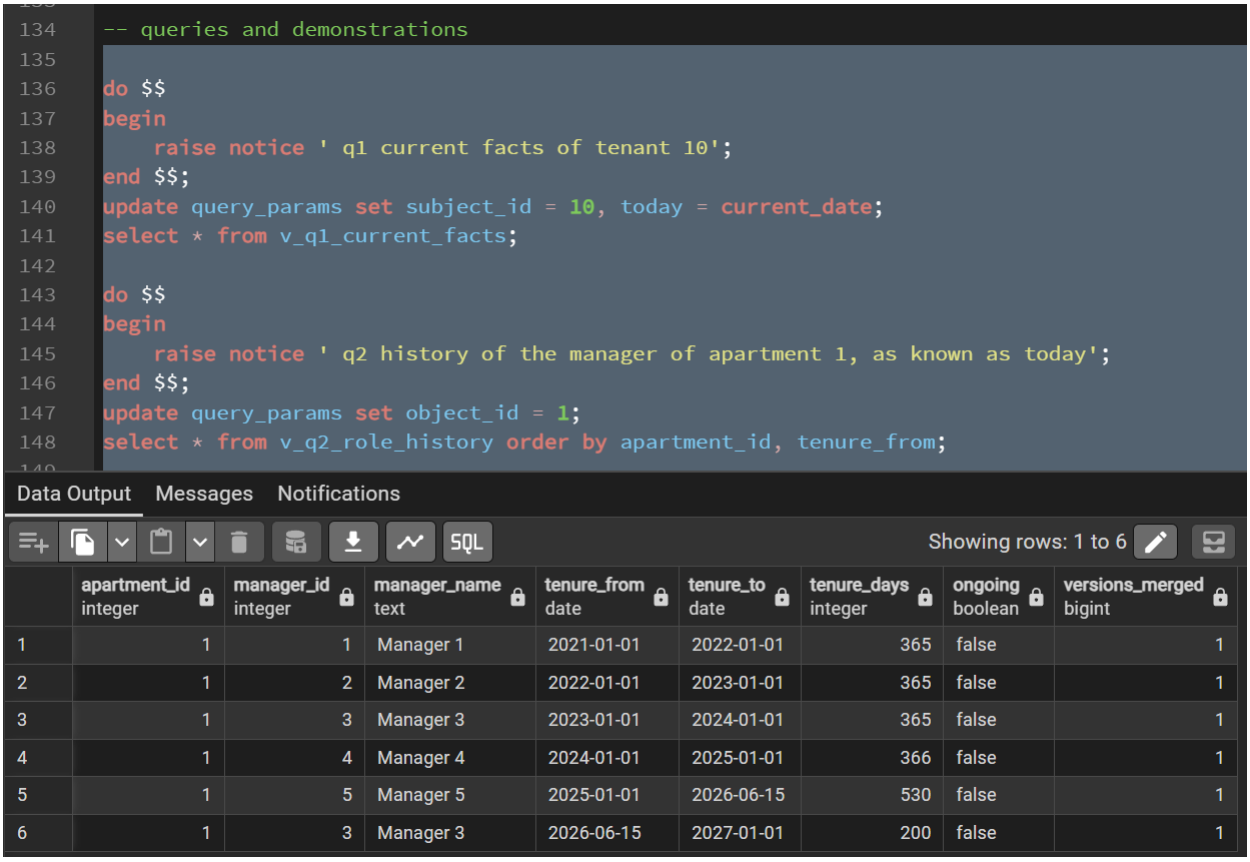
Data Output | Messages | Notifications

Showing rows: 1 to 4 | Page No: 1 of

	invariant text	table_name text	business_key text	detail text
1	I1	apartment_rent_version	apartment 1	valid [2015-01-01,2021-01-01) overlaps [2015-01-01,2021-01-01)
2	I2	lease_version	tenant 1 / apartment 1	lease [1990-01-01,1990-02-01) is not covered by listed-rent periods {[2015-01-01,9999-12-...
3	I3	apartment_manager_versi...	apartment 1 (managers 1 and...	valid [2021-01-01,2022-01-01) overlaps [2021-01-01,2022-01-01)
4	I4	lease_version	version_id 1	tx_to 10099-12-31 23:59:59.999999+00 is in the future

Image 3: Manager History (Q2)

Shows the history of managers assigned to apartment 1, including each manager's tenure. It addresses the problem of tracking role changes over time and presenting the history as continuous periods.



```
134  -- queries and demonstrations
135
136  do $$
137  begin
138      raise notice ' q1 current facts of tenant 10';
139  end $$;
140  update query_params set subject_id = 10, today = current_date;
141  select * from v_q1_current_facts;
142
143  do $$
144  begin
145      raise notice ' q2 history of the manager of apartment 1, as known as today';
146  end $$;
147  update query_params set object_id = 1;
148  select * from v_q2_role_history order by apartment_id, tenure_from;
```

Data Output | Messages | Notifications

Showing rows: 1 to 6

	apartment_id integer	manager_id integer	manager_name text	tenure_from date	tenure_to date	tenure_days integer	ongoing boolean	versions_merged bigint
1	1	1	Manager 1	2021-01-01	2022-01-01	365	false	1
2	1	2	Manager 2	2022-01-01	2023-01-01	365	false	1
3	1	3	Manager 3	2023-01-01	2024-01-01	365	false	1
4	1	4	Manager 4	2024-01-01	2025-01-01	366	false	1
5	1	5	Manager 5	2025-01-01	2026-06-15	530	false	1
6	1	3	Manager 3	2026-06-15	2027-01-01	200	false	1

Image 4: Coalescing Adjacent Periods (Q2)

Compares raw manager-version records with the aggregated history. Two adjacent records for Manager 3 are merged into one continuous period, avoiding unnecessary fragmentation in

temporal reports.

```
do $$
begin
    raise notice ' raw current rows (the last tenure is split in two adjacent rows)';
end $$;
select version_id, manager_id, valid_from, valid_to
from apartment_manager_version
where apartment_id = 1 and tx_to = timestamptz '9999-12-31 23:59:59.999999+00' order by valid_from;

do $$
begin
    raise notice ' q2 over the same data: adjacent same-holder rows are coalesced';
end $$;
select * from v_q2_role_history order by apartment_id, tenure_from;
```

Data Output | Messages | Notifications

Showing rows: 1 to 6 | Page No:

	apartment_id integer	manager_id integer	manager_name text	tenure_from date	tenure_to date	tenure_days integer	ongoing boolean	versions_merged bigint
1	1	1	Manager 1	2021-01-01	2022-01-01	365	false	1
2	1	2	Manager 2	2022-01-01	2023-01-01	365	false	1
3	1	3	Manager 3	2023-01-01	2024-01-01	365	false	1
4	1	4	Manager 4	2024-01-01	2025-01-01	366	false	1
5	1	5	Manager 5	2025-01-01	2026-06-15	530	false	1
6	1	3	Manager 3	2026-06-15	2027-01-01	200	false	2

Image 5: Gaps Between Lease Periods (Q8)

Identifies gaps between consecutive lease periods for the same tenant and apartment. It helps detect periods when an apartment may have been vacant or when lease history may be incomplete.

```
do $$
begin
    raise notice ' q8 gaps between valid periods of the same pair (tenant, apartment)';
end $$;
select * from v_q8_gaps order by tenant_id, apartment_id, left_on;
```

Data Output | Messages | Notifications

Showing rows: 1 to 3

	tenant_id integer	tenant_name text	apartment_id integer	left_on date	returned_on date	gap_days integer
1	10	Tenant 10	1	2026-04-...	2026-05-10	30
2	20	Tenant 20	2	2026-04-...	2026-05-12	27
3	30	Tenant 30	3	2026-04-...	2026-05-15	25

Conclusion

The project demonstrates how bitemporal modeling preserves both the real-world history of rental facts and the history of the database's knowledge. PostgreSQL's constraints protect important invariants, while the views and write protocol provide the historical queries and

correction semantics needed by the domain. The EXPLAIN plans and final Q0–Q8 outputs are the evidence required to complete the performance and correctness evaluation.
